

Resultados - Metodologia Científica

Bernardo M. Detomi; Leonardo P. Johansson; Matheus Barbosa

29 de setembro de 2025

1. Análise dos resultados

1.1 Casos pequenos ($n=10$ e $n=100$)

Todos os algoritmos apresentam tempos praticamente nulos (aproximadamente o segundos). Isso acontece porque listas pequenas não evidenciam a diferença entre algoritmos quadráticos ($O(n^2)$) e log-lineares ($O(n \log n)$). O uso de memória também é muito baixo (algo em torno de KB).

1.2 Casos médios ($n=1000$)

HeapSort:

Mantém bom desempenho, com tempos na faixa de 0.007s–0.015s. Escala de forma previsível com $O(n \log n)$.

InsertionSort [3]:

Excelente em entrada ordenada (0.002s), pois no melhor caso é $O(n)$. Mas perde rendimento em entrada reversa e aleatória ($\approx 0.28s$ – $0.44s$), mostrando o pior caso $O(n^2)$.

BubbleSort e SelectionSort [1] [2]:

Muito mais lentos (0.29s–0.84s), confirmando a ineficiência para n maiores. Ambos mantêm padrão $O(n^2)$ em qualquer caso.

1.3 Casos grandes ($n=10000$)

Nessa parte a diferença começa a ser maior:

HeapSort:

Escalabilidade (0.15s–0.17s). Consistente em todas as entradas. $O(n \log n)$.

InsertionSort [3]:

Muito rápido em entrada ordenada (0.02s), aproveitando o melhor caso $O(n)$. Mas em entrada reversa degrada para 70s e para 40s em aleatória → prova do pior caso quadrático.

SelectionSort [2]:

Constante em todos os tipos de entrada, mas sempre lento ($\approx 30s$). Isso reflete o comportamento sempre $O(n^2)$.

BubbleSort [1]:

O pior de todos: 61s em ordenada, 104s em aleatória, e até 138s em reversa. Mostra porque é considerado o algoritmo mais ineficiente para grandes n .

1.4 Memória

HeapSort consome um pouco mais (0.27KB → 0.75KB), mas ainda é insignificante. BubbleSort [1], SelectionSort [2] e InsertionSort [3] usam memória praticamente constante (0.1–0.28KB). Em termos de memória, todos são eficientes, a grande diferença é realmente o tempo de execução.

2. Resultado comparativo

Algoritmo	Melhor caso	Pior caso	Resultado Prático
HeapSort	$O(n \log n)$	$O(n \log n)$	Sempre rápido, consistente e escalável.
InsertionSort [3]	$O(n)$	$O(n^2)$	Muito eficiente para listas pequenas ou já ordenadas, mas inviável em grandes listas desordenadas.
SelectionSort [2]	$O(n^2)$	$O(n^2)$	Sempre lento, não aproveita listas ordenadas.
BubbleSort [1]	$O(n)$	$O(n^2)$	Pior desempenho em quase todos os cenários, inviável para grandes n.

Figura 1: Comparação de tempo

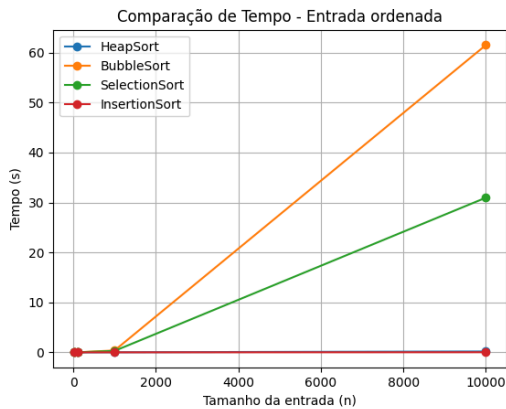


Figura 2: Comparação de tempo

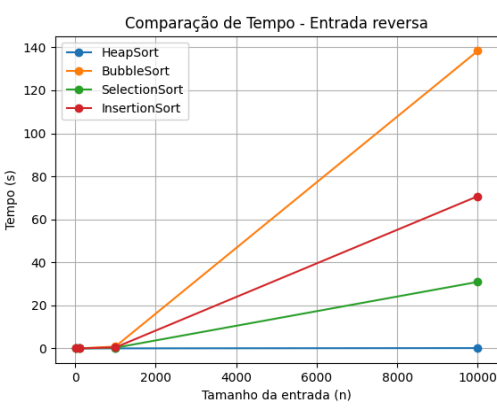


Figura 3: Comparação de tempo

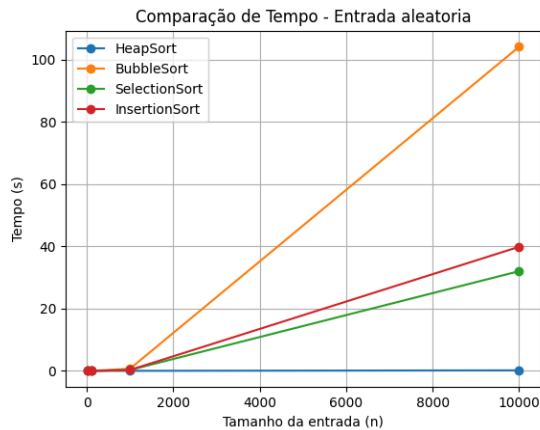


Figura 4: Comparação de Memória

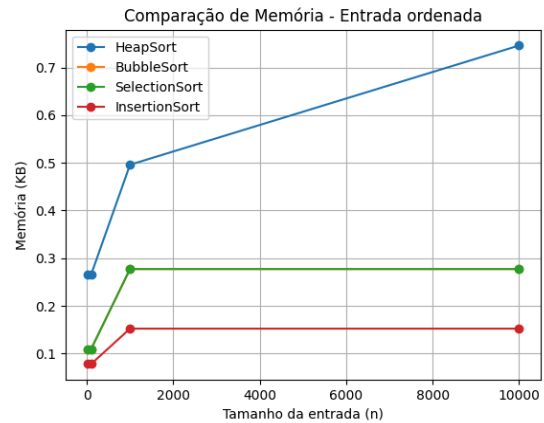


Figura 5: Comparação de memória

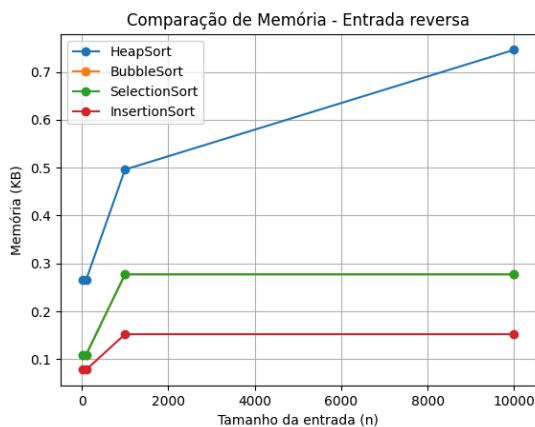
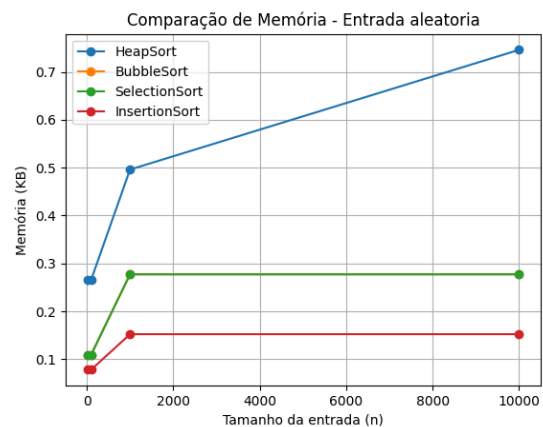


Figura 6: Comparação de memória



3. Conclusão

A análise comparativa evidenciou que, para conjuntos de dados pequenos ($n \leq 100$), todos os algoritmos avaliados apresentam desempenho semelhante, tornando a escolha do método pouco relevante. Entretanto, em tamanhos intermediários ($n \approx 1000$), o HeapSort já se destaca de forma significativa, demonstrando maior eficiência em relação aos demais. Para valores grandes ($n \geq 10.000$), o HeapSort se mostra a única alternativa realmente viável, enquanto o InsertionSort [3] pode ser considerado apenas em cenários específicos, quando a entrada se encontra quase ordenada. Já o SelectionSort [2] e o BubbleSort [1] apresentam desempenho insatisfatório em qualquer escala maior, sendo recomendada a sua exclusão de uso prático.

Referências

- [1] Bubble Sort.
- [2] Selection Sort.
- [3] Insertion Sort.

Github

<https://github.com/BernardoDetomi/Metodologia-Cientifica.git>